

Github repo:

https://github.com/AD2000X/Loan-Prediction_MLOps

0. Abstract

This report presents an end-to-end machine learning operations (MLOps) system for a loan approval prediction task deployed on the AWS platform. The project goes beyond training a single model and focuses on building a reproducible, observable and maintainable pipeline that covers data versioning, model training and tracking, model registry, containerisation, Kubernetes-based deployment, online inference, system monitoring, and data drift analysis.

Datasets are tracked with Data Version Control (DVC) and stored on Amazon S3, while experiments and models are managed through MLflow, including a central model registry. A FastAPI-based inference service is containerised with Docker and deployed on Amazon Elastic Kubernetes Service (EKS), exposing prediction and health-check endpoints. Prometheus collects metrics from the API service, and Grafana provides dashboards for real-time monitoring. Data drift is monitored using an Evidently-powered Streamlit application, which compares a reference training dataset with fresh prediction batches collected from the Kubernetes-hosted service.

The system demonstrates how a typical “Kaggle-style” loan prediction problem can be transformed into a production-grade MLOps project. The report combines architectural descriptions from the project documentation with 46 figures (1–46), which together provide a full evidence trail from datasets and training runs to online predictions and drift dashboards.

1. Introduction

1.1 Project goal

The goal of this project is not only to achieve good predictive performance on the loan approval task, but to build a complete MLOps system around it. More precisely, the system aims to:

1. **Version data and models in a reproducible way** using DVC and MLflow.
2. **Train, tune and register models** through a structured training pipeline with experiment tracking.
3. **Serve predictions via a robust API** (FastAPI) packaged in a Docker image and deployed on EKS.
4. **Monitor the system and model behaviour** with Prometheus and Grafana.
5. **Detect potential data drift** by comparing current prediction batches to a reference training dataset using Evidently.

1.2 Scope of the report

This report describes the system from the problem definition and dataset through the architecture, pipelines, deployment stack, and monitoring infrastructure.

The remainder of the report is structured as follows:

- Section 2 defines the problem and describes the datasets.
- Section 3 gives a high-level system architecture overview.
- Sections 4–5 explain data and experiment management, and the training pipeline.
- Section 6 focuses on containerisation and deployment.
- Section 7 presents the online inference service.
- Section 8 covers monitoring and observability.
- Section 9 explains the data drift monitoring design and dataset evidence.
- Section 10 organises the evidence into a clear end-to-end chain.
- Sections 11 and 12 discuss limitations, future work, and conclusions.
- The appendices contain key commands, the figure index, and supporting configuration snippets.

2. Problem Definition and Dataset

2.1 Problem statement

The core task is **loan approval prediction**. Each record represents a loan application with features describing the applicant's personal and financial situation, as well as basic loan details. The model predicts whether the loan should be approved (Loan_Status = Y) or rejected (Loan_Status = N).

This task is implemented as a supervised **binary classification** problem. The main evaluation metrics are **accuracy** and **F1-score**, with F1 used to balance precision and recall because false positives (approving high-risk applicants) and false negatives (rejecting good applicants) both have meaningful costs.

2.2 Features and target

The typical structure of the loan dataset includes:

- **Categorical features**
 - Gender (e.g. *Male, Female*)
 - Married (e.g. *Yes, No*)
 - Dependents (e.g. *0, 1, 2, 3+*)
 - Education (e.g. *Graduate, Not Graduate*)
 - Self_Employed (e.g. *Yes, No*)
 - Property_Area (e.g. *Urban, Semiurban, Rural*)
- **Numerical features**
 - ApplicantIncome
 - CoapplicantIncome
 - LoanAmount (often transformed)
 - Loan_Amount_Term
 - Credit_History (binary indicator)
- **Target**
 - Loan_Status $\in \{Y, N\}$

The project stores training and test splits in a datasets/ directory, tracked through DVC.

2.3 Dataset roles: training, test, and production batches

The project distinguishes between several dataset roles:

- **Training dataset (train.csv)**: used to train and validate models. This dataset later becomes part of the **reference** (baseline) for drift analysis.
- **Test dataset (test.csv)**: used for final offline evaluation before deployment.

- **Current production batch:** small collections of recent API requests retrieved from the Kubernetes-hosted FastAPI service. These batches are used to check whether the live data distribution has drifted away from the training distribution.

The production batch is stored in files such as `drift_monitoring/latest_batch.csv` and `drift_monitoring/k8s_latest_batch.csv`.

2.4 Example: K8s prediction batch

A key example of live data is the batch of five records collected from the deployed service. Each record is labelled with an ID starting with `K8S_` to indicate that it originates from the Kubernetes environment:

- `K8S_001, Male, Yes, 0, Graduate, No, 5000, 2000, 150, 360, 1, Urban, Y`
- ...
- `K8S_005, Female, Yes, 1, Graduate, Yes, 4500, 2500, 180, 360, 1, Urban, Y`

These five rows form the **current batch** used in drift analysis and are shown in **Figure 1**, which displays the `latest_batch.csv` file output from the system.

```
(venv) PS C:\Users\chang\Loan-Prediction_MLOps> Get-Content drift_monitoring\latest_batch.csv
Loan_ID,Gender,Married,Dependents,Education,Self_Employed,ApplicantIncome,CoapplicantIncome,LoanAmount,Loan_Amount_Term,Credit_History,Property_Area,Loan_Status
K8S_001,Male,Yes,0,Graduate,No,5000,2000,150,360,1,Urban,Y
K8S_002,Female,No,1,Graduate,Yes,4000,0,100,360,1,Rural,Y
K8S_003,Male,Yes,2,Not Graduate,No,3000,1500,120,360,0,Semiurban,N
K8S_004,Male,No,0,Graduate,No,6000,0,200,360,1,Urban,Y
K8S_005,Female,Yes,1,Graduate,Yes,4500,2500,180,360,1,Urban,Y
```

Figure 1. Preview of `latest_batch.csv` collected from the Kubernetes-hosted prediction service (IDs `K8S_001`–`K8S_005`).

3. System Architecture Overview

3.1 End-to-end MLOps pipeline

The overall architecture can be summarised as a seven-phase MLOps pipeline:

workflow

1. **Data versioning** (DVC + S3)
2. **Training pipeline** (MLflow)
3. **Model registry and staging** (MLflow Model Registry)
4. **Deployment and serving** (Docker + Kubernetes on EKS)
5. **API service** (FastAPI)
6. **Metrics collection** (Prometheus)
7. **Drift detection** (Evidently via Streamlit)

Data flows from local datasets tracked by DVC into the training pipeline, where models are trained and logged to MLflow. Candidate models are registered and promoted through stages (None → Staging → Production → Archived).

workflow

The selected production model is packaged in a Docker image and deployed to EKS. The FastAPI service exposes prediction and metrics endpoints. Prometheus scrapes metrics, and Grafana visualises them. Finally, data collected from the online service is analysed in Evidently to detect drift.

3.2 AWS platform and components

The system uses several AWS services:

- **Amazon S3:** stores datasets and MLflow artefacts (e.g. s3://loanpred-mlops-20251118-120330).
- **AWS ECR:** hosts Docker images for the loan prediction service.
- **Amazon EKS:** runs the Kubernetes cluster that hosts the FastAPI deployment and associated monitoring stack.
- **IAM / IRSA:** grants pods access to S3 and other resources through a service account and IAM role.

3.3 Technology stack

The main technology components include:

- **Data and versioning:** DVC, AWS S3
- **Experiment tracking and registry:** MLflow
- **Modelling:** scikit-learn, XGBoost, LightGBM, Hyperopt
- **Inference API:** FastAPI, Uvicorn
- **Containerisation and orchestration:** Docker, Kubernetes (EKS)

- **Monitoring:** Prometheus, Grafana
 - **Drift monitoring:** Evidently + Streamlit
 - **CI/CD:** GitHub Actions
-

4. Data and Experiment Management

4.1 Data versioning with DVC and S3

Raw loan data is stored under the `datasets/` directory, typically as `train.csv` and `test.csv`. DVC is used to track changes to these files and to manage synchronisation with an S3 bucket:

- `dvc add datasets/` creates `.dvc` files that store metadata about the datasets.
- `dvc push` uploads the actual data to the DVC remote (S3).
- `dvc pull` downloads the tracked data when needed, such as in CI/CD jobs or on a new machine.

The `.dvc/config` specifies the remote S3 bucket:

```
remote = myremote
url = s3://loanpred-mlops-20251118-120330
region = eu-west-2
```

This integration ensures that any team member who checks out a given Git commit and runs `dvc pull` obtains the exact dataset version used for that commit.

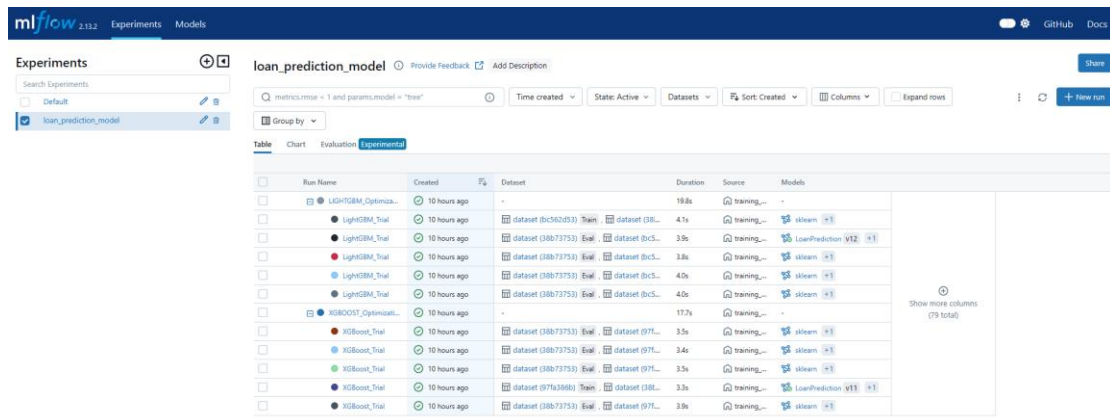
4.2 MLflow experiment tracking

MLflow is the central tool for logging experiments, metrics, parameters, and artefacts. The MLflow UI is typically served via:

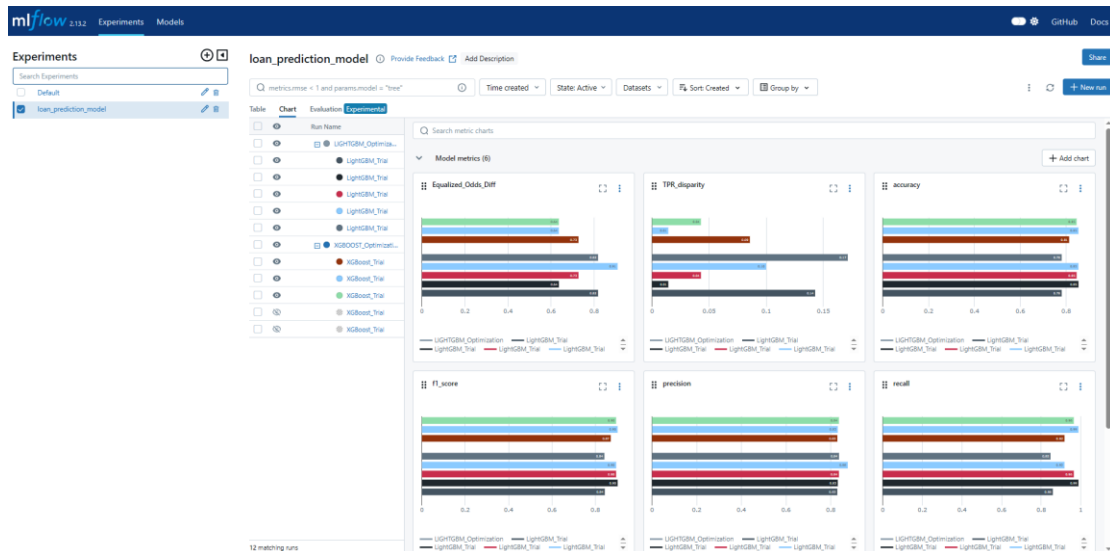
```
mlflow ui --backend-store-uri file:./mlruns --port 5000
```

Loan-Prediction_MLOps

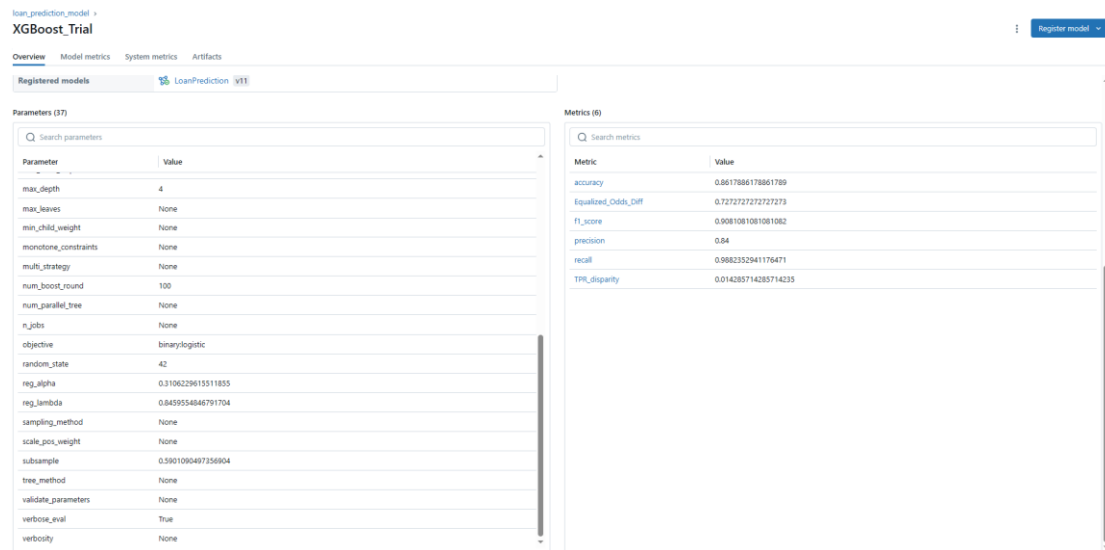
The following figures from the MLflow UI are included in the report:



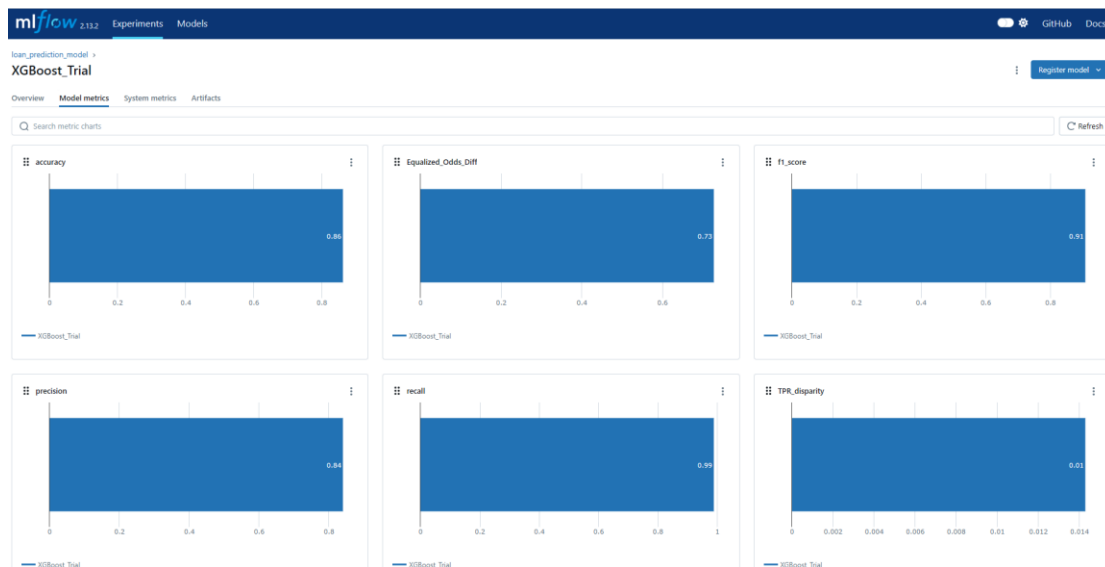
• **Figure 2.** MLflow experiment list view for loan_prediction_model.



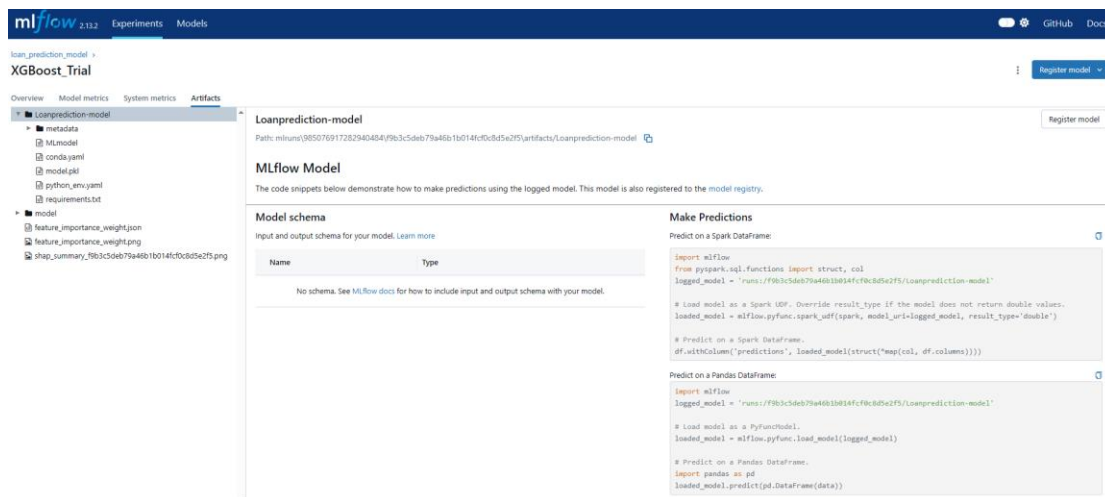
• **Figure 3.** Run list for a selected experiment, showing run IDs, start times, and main metrics.



- **Figure 6.** Run detail view – *Artifacts* tab, including model files and plots.



- **Figure 7.** Comparison view of multiple runs, highlighting differences in F1 and accuracy.



- **Figure 8.** Example SHAP or fairness-related artefact logged in MLflow.

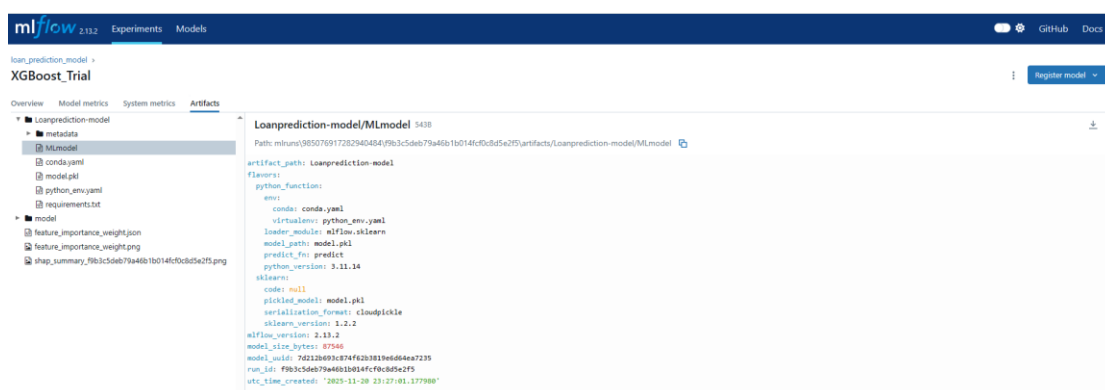
These figures show how each training run is fully documented and comparable.

4.3 Model registry and stage management

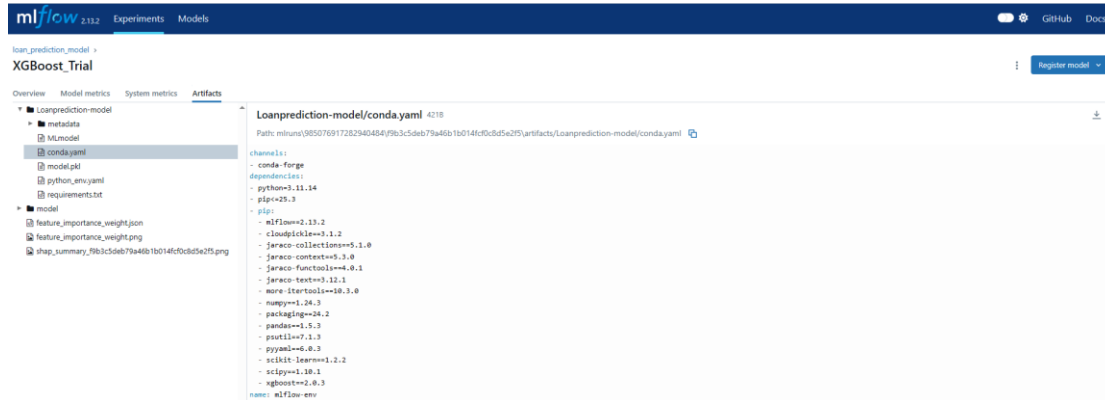
The project uses the **MLflow Model Registry** to manage the lifecycle of the main model, usually named something like "LoanPrediction". The registry supports stage transitions, as illustrated by the typical API usage:

```
mlflow.MlflowClient().transition_model_version_stage(
    name="LoanPrediction",
    version=2,
    stage="Production" # None -> Staging -> Production -> Archived
)
```

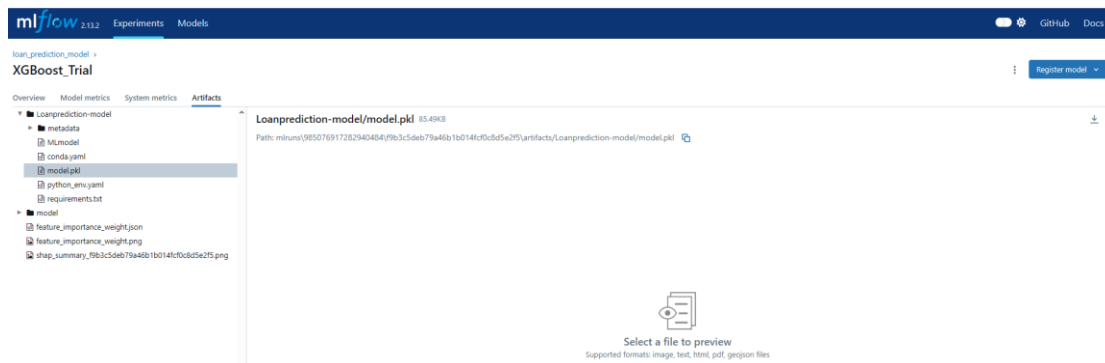
The following registry-related figures are used:



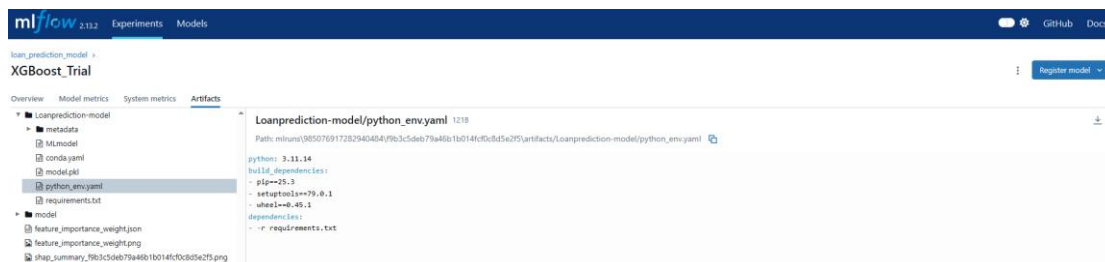
- **Figure 9.** Model Registry list showing all versions of LoanPrediction.



- **Figure 10.** Model version detail page (e.g. Version 11, XGBoost, Production).



- **Figure 11.** Stage transition history for a model version (None $\rightarrow$ Staging $\rightarrow$ Production).



- **Figure 12.** Overview of registered models with their current stages and latest metrics.

These figures provide visual proof that the deployed model originates from a properly registered and staged version in MLflow.

5. Training and Evaluation Pipeline

5.1 Preprocessing and feature engineering

The training pipeline is built around a preprocessing and feature engineering module. The pipeline consists of steps such as:

- `ModeImputer` and `MeanImputer` for handling missing categorical and numerical values.
- `DropFeatures` for removing unused or leaky columns.
- `LogTransform` for skewed features like `LoanAmount`.
- `CustomLabelEncoder` for categorical variables.

These steps are organised into a scikit-learn style pipeline inside `prediction_model/processing/preprocessing.py`, which ensures that both training and inference share exactly the same transformations.

5.2 Automated training pipeline

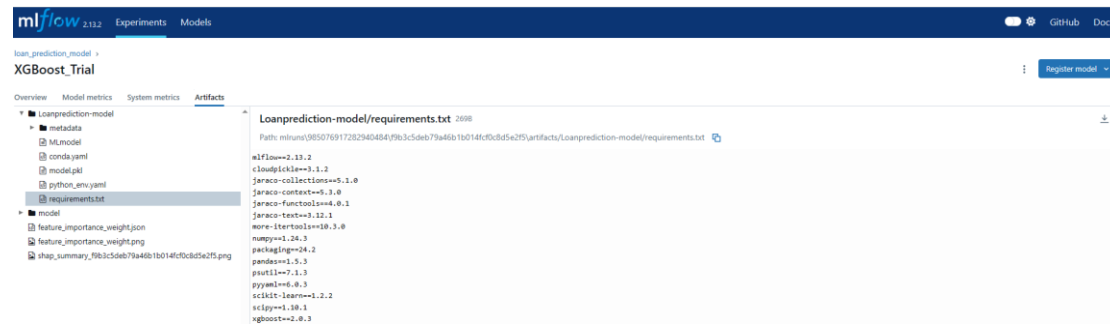
The main training script, `prediction_model/training_pipeline.py`, orchestrates the full training process:

1. Load `datasets/train.csv` and `datasets/test.csv`.
2. Split data into training and validation sets if needed.
3. Apply the preprocessing pipeline.
4. Train candidate models, such as `XGBoost` and `LightGBM`.
5. Use `Hyperopt` to search over hyperparameters for each model family.
6. Log parameters, metrics, and artefacts to MLflow.
7. Optionally register the best-performing model in the Model Registry.

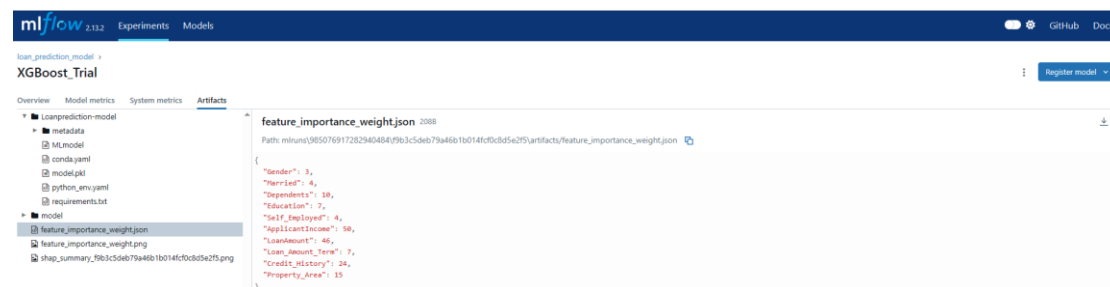
This automated pipeline is used both locally and inside CI/CD jobs, ensuring that the same code path runs in all environments.

5.3 Model selection and performance

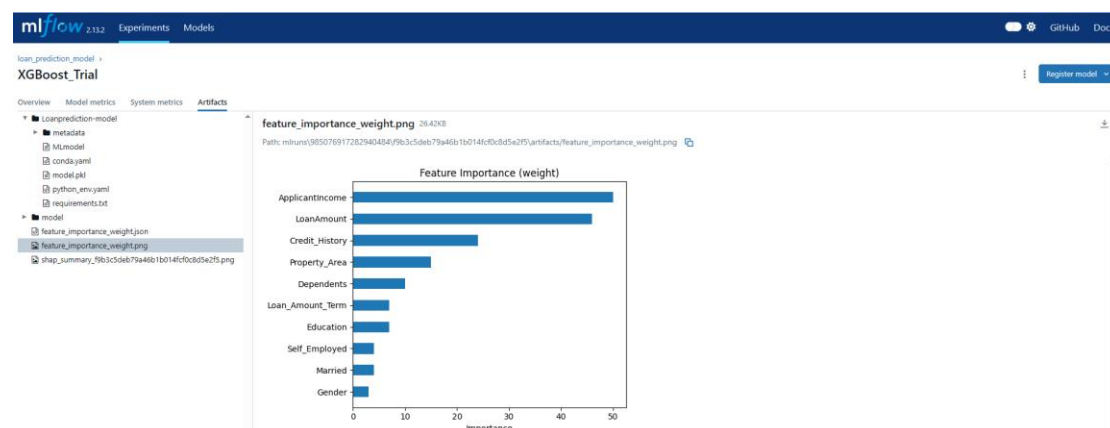
Model performance is evaluated using metrics such as accuracy and F1-score. The MLflow runs are compared to decide which model should be promoted to the Production stage. Typical plots and views include:



- **Figure 13.** MLflow run comparison plot for F1-score across different hyperparameter configurations.



- **Figure 14.** Accuracy comparison across models (e.g. XGBoost vs LightGBM).



- **Figure 15.** ROC or precision–recall curve for the best model.



- **Figure 16.** SHAP summary plot for feature importance.

The screenshot displays the mlflow web interface for the 'Models' tab. It shows a table of registered models for the 'LoanPrediction' experiment. The table has columns for Name, Latest version, Staging, Production, Created by, Last modified, and Tags. A single model is listed with the name 'LoanPrediction', latest version 'Version 12', staging 'Version 12', production 'Version 11', created by 'mlflow', last modified '2025-11-21 10:31:43', and no tags.

Name	Latest version	Staging	Production	Created by	Last modified	Tags
LoanPrediction	Version 12	Version 12	Version 11	mlflow	2025-11-21 10:31:43	—

- **Figure 17.** Fairness-related plot or metric summary (if logged).

Registered Models > LoanPrediction

Created Time: 2025-11-18 14:24:09 Last Modified: 2025-11-21 10:31:43

> Description [Edit](#)

> Tags

▼ Versions All Active 3 Compare New model registry UI

Version	Registered at	Created by	Stage	Description
Version 12	2025-11-20 23:27:32		Staging	LIGHTGBM model trained on 2025...
Version 11	2025-11-20 23:27:12		Production	XGBOOST model trained on 2025...
Version 10	2025-11-20 23:22:25		Archived	LIGHTGBM model trained on 2025...
Version 9	2025-11-20 23:22:06		Archived	XGBOOST model trained on 2025...
Version 8	2025-11-20 22:48:43		Archived	LIGHTGBM model trained on 2025...
Version 7	2025-11-20 22:48:23		Archived	XGBOOST model trained on 2025...
Version 6	2025-11-20 22:27:06		Archived	LIGHTGBM model trained on 2025...
Version 5	2025-11-20 22:26:46		Archived	XGBOOST model trained on 2025...
Version 4	2025-11-20 22:05:58		Archived	LIGHTGBM model trained on 2025...
Version 3	2025-11-20 22:05:38		Archived	XGBOOST model trained on 2025...

1 2 >

- Figure 18.** Detailed metrics table emphasising F1 and accuracy.

Registered Models > LoanPrediction > Version 11

Registered At: 2025-11-20 23:27:12 Last Modified: 2025-11-20 23:27:13 Source Run: XGBoost_Trial Stage: Production

> Description [Edit](#)

XGBOOST model trained on 2025-11-20. F1 Score: 0.9081. Accuracy: 0.8618

▼ Tags

Name	Value	Actions
f1_score	0.9081081081082	Edit Delete
model_type	xgboost	Edit Delete
training_date	2025-11-20 23:27:12	Edit Delete

Name Value

▼ Schema

Name	Type
------	------

No schema. See [MLflow docs](#) for how to include input and output schema with your model.

- Figure 19.** Training loss or early stopping curves for a selected model.

Registered Models > LoanPrediction > Version 12

Registered At: 2025-11-20 23:27:32 Last Modified: 2025-11-20 23:27:32 Source Run: Lightgbm_Trial Stage: Staging

> Description [Edit](#)

LIGHTGBM model trained on 2025-11-20. F1 Score: 0.9032. Accuracy: 0.8537

▼ Tags

Name	Value	Actions
f1_score	0.90325806451613	Edit Delete
model_type	lightgbm	Edit Delete
training_date	2025-11-20 23:27:32	Edit Delete

Name Value

▼ Schema

Name	Type
------	------

No schema. See [MLflow docs](#) for how to include input and output schema with your model.

- **Figure 20.** Final run detail highlighting the chosen production model version.

These figures collectively support the selection of the production model used in deployment.

6. Containerisation and Deployment on AWS

6.1 Docker image design

The project uses a Dockerfile that builds a production image for the FastAPI service. The build process can be summarised as:

1. Start from python:3.11-slim.
2. Install system dependencies (including AWS CLI if required).
3. Copy requirements.txt and install Python dependencies.
4. Copy main.py and the prediction_model/ package.
5. Add a start.sh script that:
 - synchronises mlruns/ from the S3 bucket,
 - loads the MLflow Model Registry, and
 - starts Uvicorn on port 8005.
6. Expose port 8005 and define a health check.

The resulting image is pushed to an AWS ECR repository, for example:

513348493761.dkr.ecr.eu-west-2.amazonaws.com/mlops/loan_pred.

workflow

6.2 Kubernetes / EKS deployment

Kubernetes manifests located in the k8s/ directory define:

- A **namespace**, e.g. loan-prediction-mlops.
- A **Deployment** with multiple replicas (e.g. three) using the ECR image and a rolling update strategy.
- A **Service** of type ClusterIP exposing the API on port 8005.
- A **ServiceAccount** with an IRSA-linked IAM role for S3 access.

The architecture diagram in the workflow file shows how external clients send requests to the Service, which routes traffic to the running Pods.

6.3 CI/CD with GitHub Actions

GitHub Actions workflows in `.github/workflows/` implement CI/CD. The typical pipeline:

1. Runs tests (including API and integration tests, if enabled).
2. Builds a Docker image and pushes it to ECR.
3. Applies Kubernetes manifests (e.g. using `kubectl apply`) to update the deployment on EKS.
4. Optionally runs smoke tests and monitors metrics post-deployment.

This ensures that changes in the repository automatically trigger rebuilding and redeployment of the service, using DVC to guarantee consistent access to the datasets.

7. Online Inference Service

7.1 FastAPI endpoints

The FastAPI app, defined in `main.py` and related modules, exposes several endpoints:

- GET / – API documentation and endpoints list.
- GET /health – health check used by Kubernetes probes.
- GET /metrics – Prometheus metrics endpoint.
- GET /model_info – returns information about the current model (version, type, stage).
- GET /list_models – lists registered model versions.
- POST /prediction_api – single-loan prediction endpoint.
- POST /batch_prediction – CSV batch prediction endpoint.

A typical JSON request body for `/prediction_api` is:

workflow

```
{  
  "Gender": "Male",
```

```
"Married": "Yes",
"Dependents": "1",
"Education": "Graduate",
"Self_Employed": "No",
"ApplicantIncome": 5000,
"CoapplicantIncome": 2000,
"LoanAmount": 150,
"Loan_Amount_Term": 360,
"Credit_History": 1,
"Property_Area": "Urban"
}
```

The request is parsed into a DataFrame, transformed by the preprocessing pipeline, passed to the model for prediction, and then returned as JSON.

7.2 Example prediction and K8s logs

A typical call to the prediction endpoint (after port-forwarding) is:

```
kubectrl port-forward -n loan-prediction-mlops svc/loan-prediction-service 8005:8005
```

```
curl.exe -X POST http://localhost:8005/prediction_api \
-H "Content-Type: application/json" \
-d "@test_request.json"
```

The response contains a prediction and metadata:

Loan-Prediction_MLOps

```
{"prediction":"Y","model_version":11,"model_type":"xgboost","model_stage":"Production","timestamp":"2025-11-20T23:42:04.492348"}
```

Kubernetes pod logs confirm that the online service uses the correct model:

- [PREDICTION] Using xgboost model v11 from Production stage
- [PREDICTION] Model metrics - F1: 0.9081, Accuracy: 0.8618
- Prediction: Y | Model: xgboost v11 (Production)
- INFO: 127.0.0.1:56140 - "POST /prediction_api HTTP/1.1" 200 OK

```
(venv) PS C:\Users\chang\Loan-Prediction_MLOps> Get-Content k8s_logs.txt | Select-String "PREDICTION|POST.*prediction_api"

Registry source path: /home/runner/work/Loan-Prediction_MLOps/Loan-Prediction_MLOps/mlruns/985076917282940484/f9b3c5deb79a46b1b014fcf0c8d5e2f5/artifacts/Loanprediction-model
Resolved relative path: mlruns/985076917282940484/f9b3c5deb79a46b1b014fcf0c8d5e2f5/artifacts/Loanprediction-model
/app/prediction_model/predict.py:224: FutureWarning: ``mlflow.tracking.client.MlflowClient.get_latest_versions`` is deprecated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: https://mlflow.org/docs/2.13.2/model-registry.html#migrating-from-stages
[PREDICTION] Using xgboost model v11 from Production stage
[PREDICTION] Model metrics - F1: 0.9081, Accuracy: 0.8618
2025-11-21 00:11:08,996 [INFO] __main__ - Prediction: Y | Model: xgboost v11 (Production)
INFO: 127.0.0.1:56140 - "POST /prediction_api HTTP/1.1" 200 OK
/app/prediction_model/predict.py:70: FutureWarning: ``mlflow.tracking.client.MlflowClient.get_latest_versions`` is deprecated since 2.9.0. Model registry stages will
```

These log lines are captured visually in **Figure 29**.

Figure 29. Kubernetes pod log excerpt showing predictions produced by the MLflow Model Registry version 11 (XGBoost, Production stage).

8. Monitoring and Observability

8.1 Metrics instrumentation

The FastAPI service integrates `prometheus_fastapi_instrumentator` to expose metrics at `/metrics`. The exported metrics include:

- `http_requests_total{method, endpoint, status}` – HTTP request counts.
- `http_request_duration_seconds{method, endpoint}` – request latency.
- `http_requests_in_progress` – current in-progress requests.
- `model_predictions_total` – count of model predictions.
- Generic process metrics such as `process_cpu_seconds_total`, `process_resident_memory_bytes`, and `python_info`.

8.2 Prometheus

Prometheus collects metrics from the FastAPI service and other components. It is deployed via docker-compose for local monitoring or through kube-prometheus-stack on EKS.

The typical Prometheus server configuration includes a scrape job like:

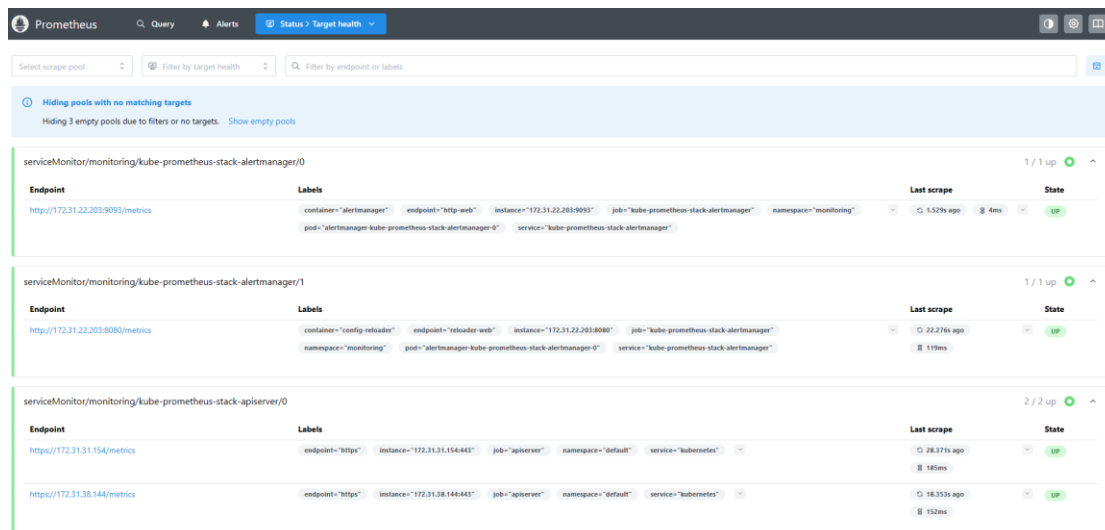
scrape_configs:

- job_name: 'loan-prediction'
- static_configs:
 - targets: ['host.docker.internal:8005']
- metrics_path: '/metrics'

In practice, queries such as the following are used:

- `model_predictions_total` – total count of predictions.
- `rate(model_predictions_total[5m])` – prediction rate over a five-minute window.
- `http_requests_total{path="/prediction_api"}` – total API requests for the prediction endpoint.
- `up{namespace="loan-prediction-mlops"}` – health of pods in the namespace.

The Prometheus figures included are:



- **Figure 21.** Prometheus UI homepage with targets overview.

Prometheus

Query

Alerts

Status > Target health

serviceMonitor/monitoring/kube-prometheus-stack-coresdns/0

2 / 2 up

Endpoint	Labels	Last scrape	State
http://172.31.23.214:9153/metrics	container="coredns" endpoint="http-metrics" instance="172.31.23.214:9153" job="coredns" namespace="kube-system" pod="coredns-f57454b8-wgpc8" service="kube-prometheus-stack-coresdns"	18.901s ago 3ms	UP
http://172.31.26.25:9153/metrics	container="coredns" endpoint="http-metrics" instance="172.31.26.25:9153" job="coredns" namespace="kube-system" pod="coredns-f57454b8-wt9p6" service="kube-prometheus-stack-coresdns"	18.411s ago 4ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-grafana/0

1 / 1 up

Endpoint	Labels	Last scrape	State
http://172.31.40.86:3000/metrics	container="grafana" endpoint="http-web" instance="172.31.40.86:3000" job="kube-prometheus-stack-grafana" namespace="monitoring" pod="kube-prometheus-stack-grafana-554f466455-qd82d" service="kube-prometheus-stack-grafana"	7.457s ago 13ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-kube-proxy/0

2 / 2 up

Endpoint	Labels	Last scrape	State
http://172.31.18.119:10249/metrics	endpoint="http-metrics" instance="172.31.18.119:10249" job="kube-proxy" namespace="kube-system" pod="kube-proxy-7y92a" service="kube-prometheus-stack-kube-proxy"	5.126s ago 6ms	UP
http://172.31.46.110:10249/metrics	endpoint="http-metrics" instance="172.31.46.110:10249" job="kube-proxy" namespace="kube-system" pod="kube-proxy-pd9fd" service="kube-prometheus-stack-kube-proxy"	16.875s ago 5ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-kube-state-metrics/0

1 / 1 up

- **Figure 22.** Query view for `model_predictions_total`.

Prometheus

Query

Alerts

Status Target health

serviceMonitor/monitoring/kube-prometheus-stack-kube-state-metrics/0

1 / 1 up

Endpoint	Labels	Last scrape	State
http://172.31.24.215:8080/metrics	container="kube-state-metrics" endpoint="http" instance="172.31.24.215:8080" job="kube-state-metrics" namespace="monitoring" pod="kube-prometheus-stack-kube-state-metrics-78f255b86-q6982" service="kube-prometheus-stack-kube-state-metrics"	28.188s ago 9ms	up

serviceMonitor/monitoring/kube-prometheus-stack-kubelet/0

2 / 2 up

Endpoint	Labels	Last scrape	State
https://172.31.18.119:10250/metrics	endpoint="https-metrics" instance="172.31.18.119:10250" job="kubelet" metrics_path="/metrics" namespace="kube-system" node="ip-172-31-18-119.eu-west-2.compute.internal" service="kube-prometheus-stack-kubelet"	21.027s ago 41ms	up
https://172.31.46.110:10250/metrics	endpoint="https-metrics" instance="172.31.46.110:10250" job="kubelet" metrics_path="/metrics" namespace="kube-system" node="ip-172-31-46-110.eu-west-2.compute.internal" service="kube-prometheus-stack-kubelet"	24.394s ago 39ms	up

serviceMonitor/monitoring/kube-prometheus-stack-kubelet/1

2 / 2 up

Endpoint	Labels	Last scrape	State
https://172.31.18.119:10250/metrics/cadvisor	endpoint="https-metrics" instance="172.31.18.119:10250" job="kubelet" metrics_path="/metrics/cadvisor" namespace="kube-system" node="ip-172-31-18-119.eu-west-2.compute.internal" service="kube-prometheus-stack-kubelet"	4.538s ago 69ms	up
https://172.31.46.110:10250/metrics/cadvisor	endpoint="https-metrics" instance="172.31.46.110:10250" job="kubelet" metrics_path="/metrics/cadvisor" namespace="kube-system" node="ip-172-31-46-110.eu-west-2.compute.internal" service="kube-prometheus-stack-kubelet"	5.716s ago 52ms	up

serviceMonitor/monitoring/kube-prometheus-stack-kubelet/2

2 / 2 up

Endpoint	Labels	Last scrape	State
----------	--------	-------------	-------

- **Figure 23.** Query view for `rate(model_predictions_total[5m])`.

Prometheus

Query

Alerts

Status > Target health

serviceMonitor/monitoring/kube-prometheus-stack-kubelet/2

2 / 2 up

Endpoint	Labels	Last scrape	State
https://172.31.18.119:10250/metrics/probes	endpoint="https-metrics"instance="172.31.18.119:10250"job="kubelet"metrics_path="/metrics/probes"namespace="kube-system" node="ip-172-31-18-119-nu-west-2-compute.internal"service="kube-prometheus-stack-kubelet"	28.00h ago3ms	UP
https://172.31.46.110:10250/metrics/probes	endpoint="https-metrics"instance="172.31.46.110:10250"job="kubelet"metrics_path="/metrics/probes"namespace="kube-system" node="ip-172-31-46-110-nu-west-2-compute.internal"service="kube-prometheus-stack-kubelet"	15.304s ago3ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-operator/0

1 / 1 up

Endpoint	Labels	Last scrape	State
https://172.31.18.43:10250/metrics	container="kube-prometheus-stack"endpoint="https"instance="172.31.18.43:10250"job="kube-prometheus-stack-operator"namespace="monitoring" pod="kube-prometheus-stack-operator-7c8c56c4-kbqgq"service="kube-prometheus-stack-operator"	25.07zs ago8ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-prometheus-node-exporter/0

2 / 2 up

Endpoint	Labels	Last scrape	State
http://172.31.18.119:9100/metrics	container="node-exporter"endpoint="http-metrics"instance="172.31.18.119:9100"job="node-exporter"namespace="monitoring" pod="kube-prometheus-stack-prometheus-node-exporter-8kxsn"service="kube-prometheus-stack-prometheus-node-exporter"	19.09zs ago27ms	UP
http://172.31.46.110:9100/metrics	container="node-exporter"endpoint="http-metrics"instance="172.31.46.110:9100"job="node-exporter"namespace="monitoring" pod="kube-prometheus-stack-prometheus-node-exporter-8tfl6"service="kube-prometheus-stack-prometheus-node-exporter"	15.818s ago24ms	UP

serviceMonitor/monitoring/kube-prometheus-stack-prometheus/0

1 / 1 up

Endpoint	Labels	Last scrape	State
----------	--------	-------------	-------

- **Figure 24.** Query view for `http_requests_total{path="/prediction_api"}`.

Query

Alerts

Status > Target health

http://172.31.46.110:9100/metrics

container="node-exporter"

endpoint="http-metrics"

instance="172.31.46.110:9100"

job="node-exporter"

namespace="monitoring"

pod="kube-prometheus-stack-prometheus-node-exporter-trfq"

service="kube-prometheus-stack-prometheus-node-exporter"

15.818s ago

24ms

UP

serviceMonitor/monitoring/kube-prometheus-stack-prometheus/0

1 / 1 up

Endpoint

Labels

Last scrape

State

http://172.31.41.115:9090/metrics

container="prometheus"

endpoint="http-web"

instance="172.31.41.115:9090"

job="kube-prometheus-stack-prometheus"

namespace="monitoring"

pod="prometheus-kube-prometheus-stack-prometheus-0"

service="kube-prometheus-stack-prometheus"

19.586s ago

10ms

UP

serviceMonitor/monitoring/kube-prometheus-stack-prometheus/1

1 / 1 up

Endpoint

Labels

Last scrape

State

http://172.31.41.115:8080/metrics

container="config-reloader"

endpoint="reloader-web"

instance="172.31.41.115:8080"

job="kube-prometheus-stack-prometheus"

namespace="monitoring"

pod="prometheus-kube-prometheus-stack-prometheus-0"

service="kube-prometheus-stack-prometheus"

11.951s ago

4ms

UP

serviceMonitor/monitoring/loan-prediction/0

3 / 3 up

Endpoint

Labels

Last scrape

State

http://172.31.23.121:8005/metrics

container="loan-prediction"

endpoint="http"

instance="172.31.23.121:8005"

job="loan-prediction-service"

namespace="loan-prediction-mlops"

pod="loan-prediction-deployment-76c7945b6c-pm47q"

service="loan-prediction-service"

2.89s ago

4ms

UP

http://172.31.34.10:8005/metrics

container="loan-prediction"

endpoint="http"

instance="172.31.34.10:8005"

job="loan-prediction-service"

namespace="loan-prediction-mlops"

pod="loan-prediction-deployment-76c7945b6c-jmrvk"

service="loan-prediction-service"

13.919s ago

2ms

UP

http://172.31.41.120:8005/metrics

container="loan-prediction"

endpoint="http"

instance="172.31.41.120:8005"

job="loan-prediction-service"

namespace="loan-prediction-mlops"

pod="loan-prediction-deployment-76c7945b6c-484tp"

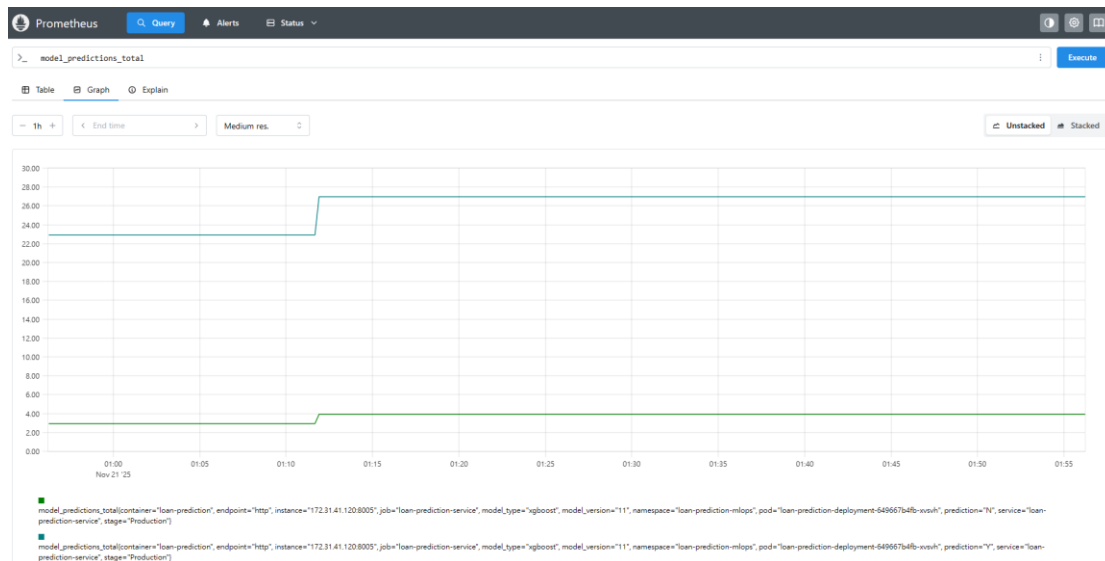
service="loan-prediction-service"

2.173s ago

2ms

UP

- **Figure 25.** Query view for `up{namespace="loan-prediction-mlops"}`.



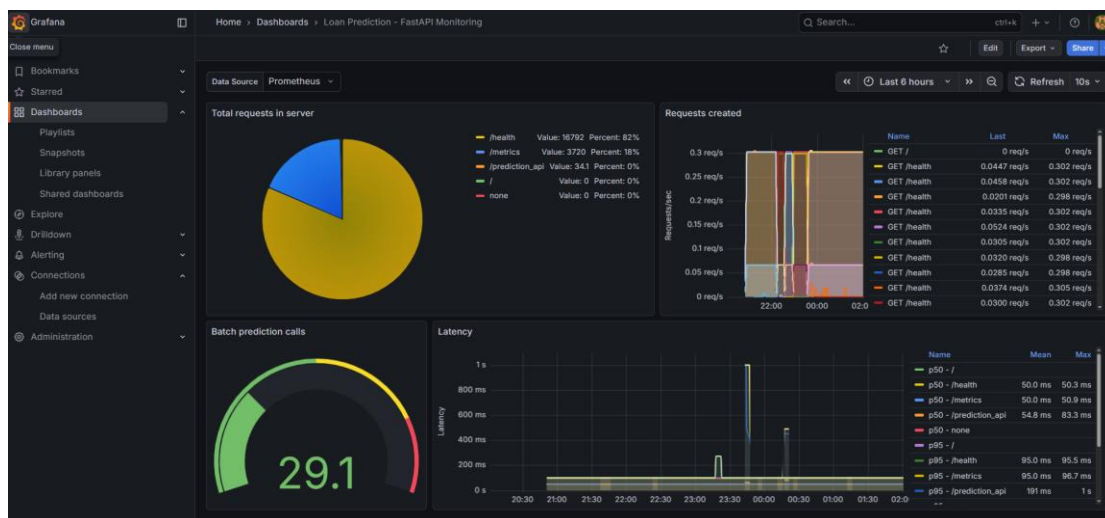
- **Figure 26.** Combined query or graph showing prediction rate over time.

8.3 Grafana

Grafana is deployed alongside Prometheus and connected as a datasource. It provides dashboards for several perspectives:

- Request rate (requests per minute).
- Latency (e.g. P95 response time).
- Error rate.
- Pod-level CPU and memory usage.
- Pod health status.

The relevant figures are:



- **Figure 27.** Grafana login or welcome screen with the Prometheus datasource configured.



- **Figure 28.** Custom loan prediction dashboard showing panels such as request rate, latency P95, error rate, CPU usage, memory usage and pod status.

These figures illustrate how operators can quickly assess the health and performance of the loan prediction service.

9. Data Drift Monitoring and Dataset Evidence

9.1 Drift monitoring architecture

Data drift monitoring is implemented using a Streamlit application powered by Evidently. The architecture is described as follows:

- **Reference data:** primarily the training dataset (train.csv) used during model development.
- **Current data:** batches of recent predictions collected from the deployed service using a helper script.
- **Evidently Streamlit app** (drift_monitoring/app_v1.py):
 - loads both datasets,

- computes drift metrics for each feature, and
- renders a dashboard showing distributions and drift status.

9.2 Collection of current batches from Kubernetes

The helper script `collect_k8s_predictions.py` sends requests to the FastAPI service and saves responses into a CSV file inside `drift_monitoring/`.

An example execution prints:

- Collecting predictions from K8s FastAPI...
- API URL: `http://localhost:8006/prediction_api`
- [1/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production
- ...
- [5/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production
- Collected 5 predictions from K8s
- Saved to: `drift_monitoring/k8s_latest_batch.csv`

This output is captured in **Figure 30**.

```
(venv) PS C:\Users\chang\Loan-Prediction_MLOps> python collect_k8s_predictions.py
● Collecting predictions from K8s FastAPI...
API URL: http://localhost:8006/prediction_api

[1/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production
[2/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production
[3/5] Prediction: N | Model: v11 (xgboost) | Stage: Production
[4/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production
[5/5] Prediction: Y | Model: v11 (xgboost) | Stage: Production

=====
Collected 5 predictions from K8s
Saved to: drift_monitoring/k8s_latest_batch.csv
=====
```

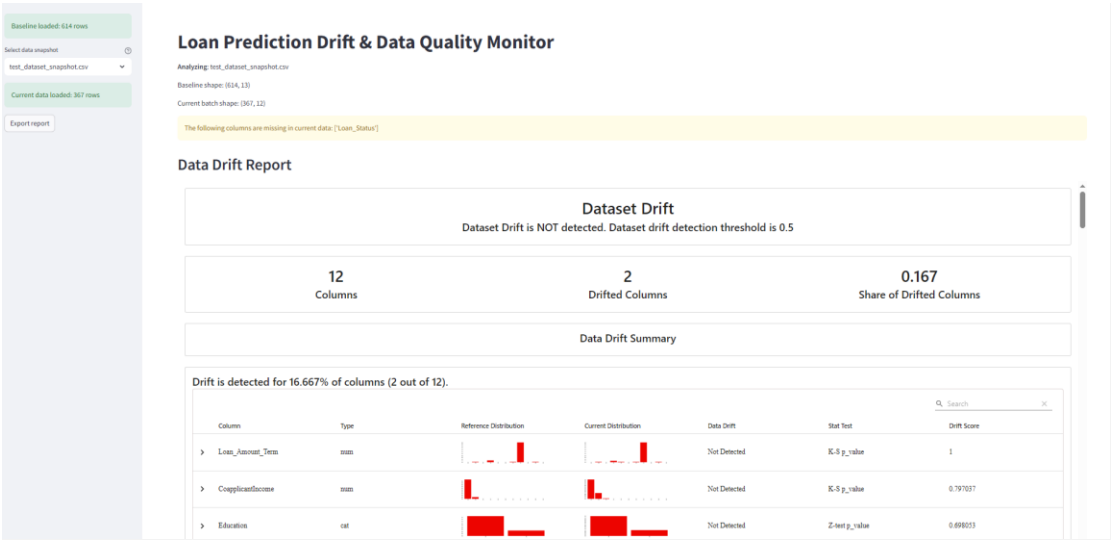
Figure 30. Console output from `collect_k8s_predictions.py`, showing five records collected from the Kubernetes-hosted FastAPI service and saved to `k8s_latest_batch.csv`.

9.3 Reference and current dataset overview

The Evidently UI shows summary information about the two datasets:

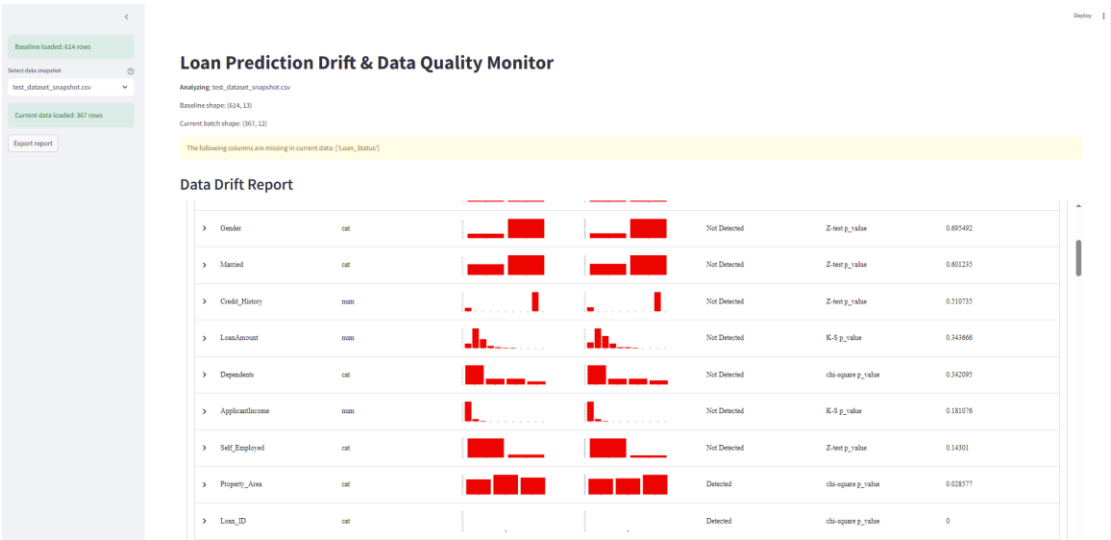
- Baseline shape: (614, 13)
- Current batch shape: (5, 13)

This confirms that the reference dataset (baseline) contains 614 rows with 13 columns, while the current batch from K8s contains 5 rows with the same number of columns. The corresponding figures is:



- **Figure 31.** Evidently dashboard header showing the message “Successfully loaded baseline and batch data” and the dataset shapes.

Another figure highlights the mapping between IDs and origin:



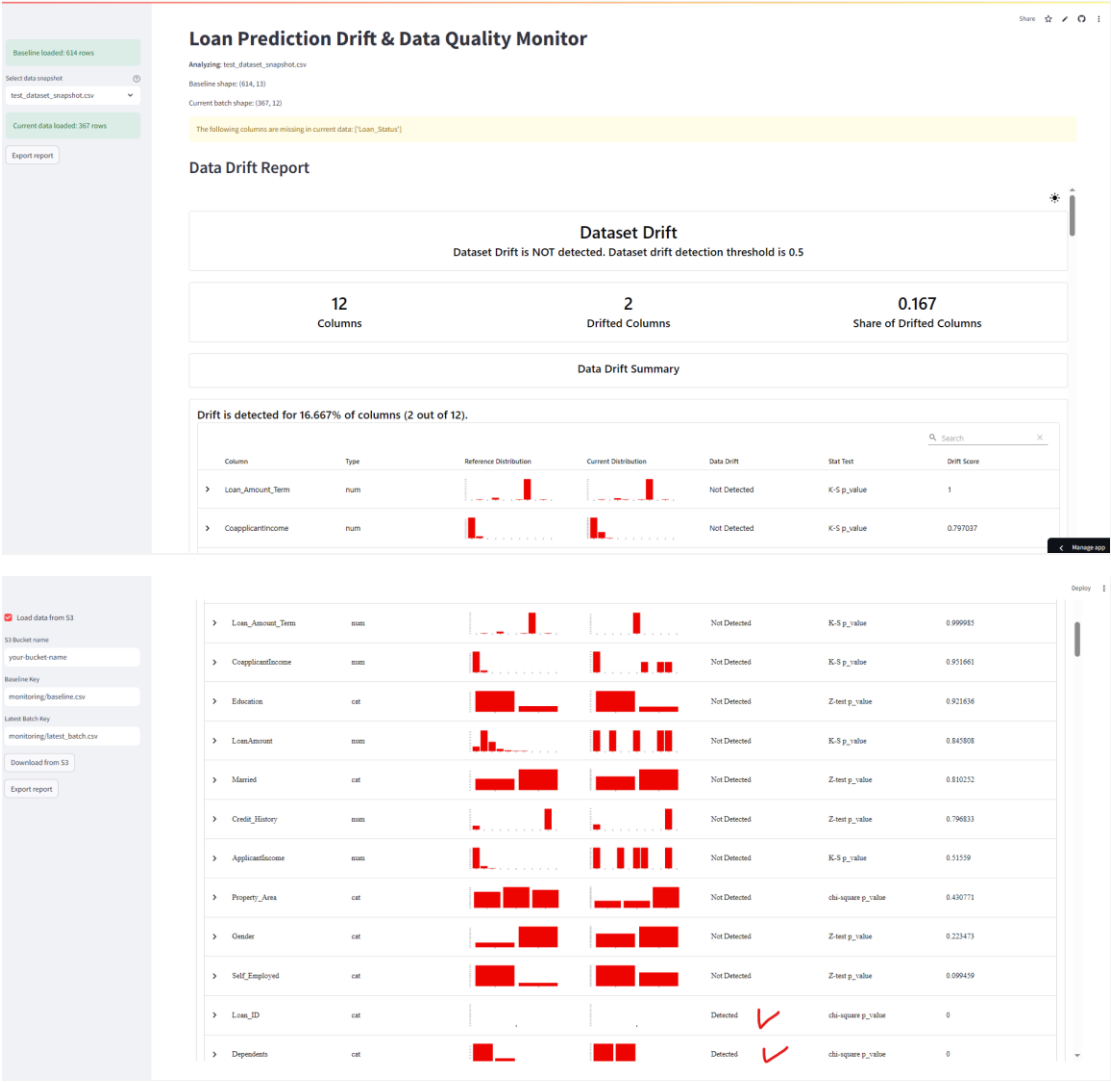
- **Figure 32.** Evidently table or summary showing that current batch records are labelled K8S_001–K8S_005, matching the contents of latest_batch.csv as shown earlier in Figure 1.

9.4 Evidently data drift report

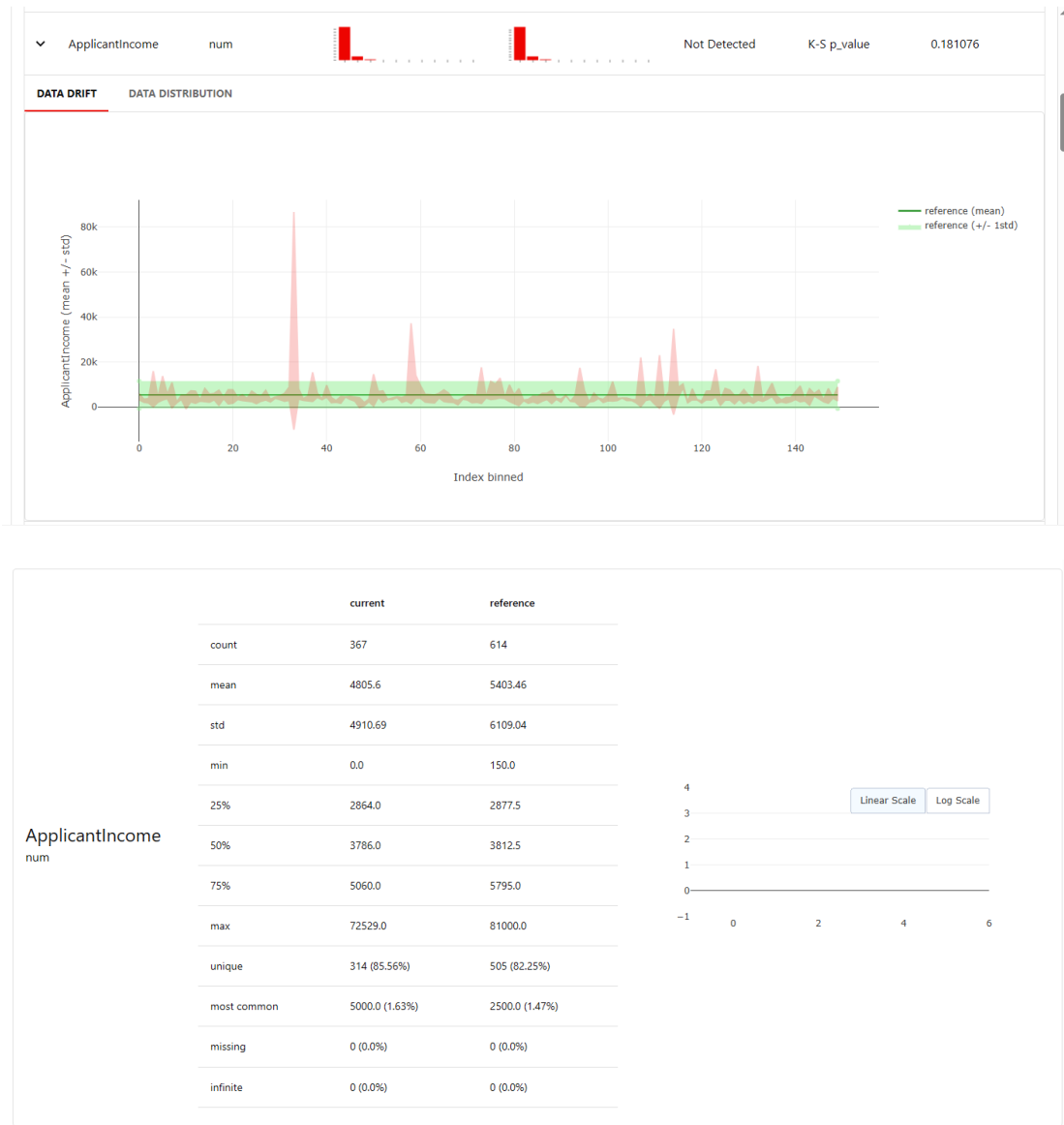
The Evidently dashboard contains multiple panels, including:

- Feature distributions for categorical variables such as Gender, Married, Education, Property_Area.
- Histograms or density plots for numerical features such as ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term.
- Drift detection results for each feature (e.g. “No drift” or “DRIFT!”).

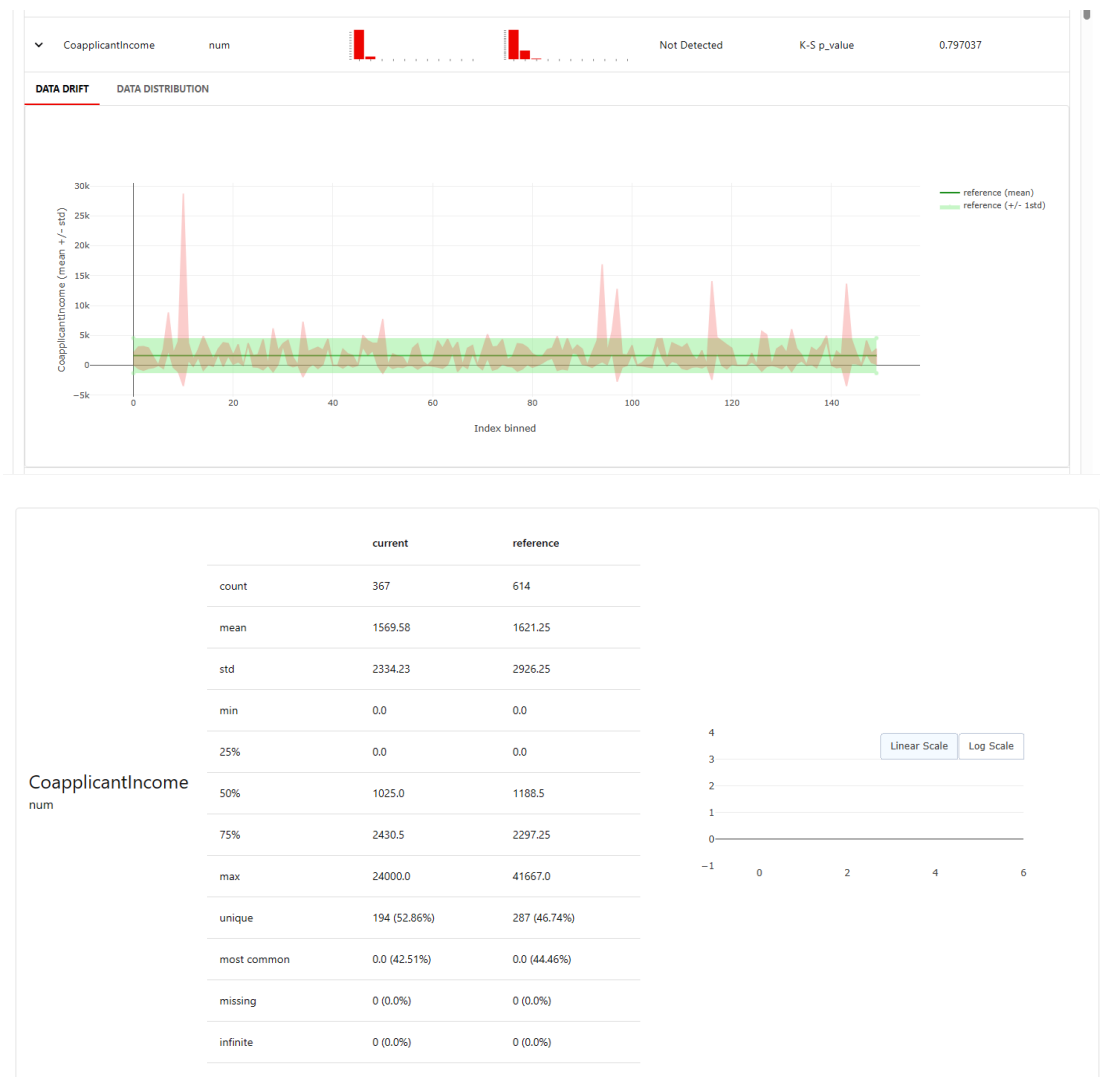
Figures used to illustrate these results are grouped as:



- **Figure 33.** Overall drift summary panel (per-feature drift flags).



- **Figure 34.** Distribution comparison for ApplicantIncome.

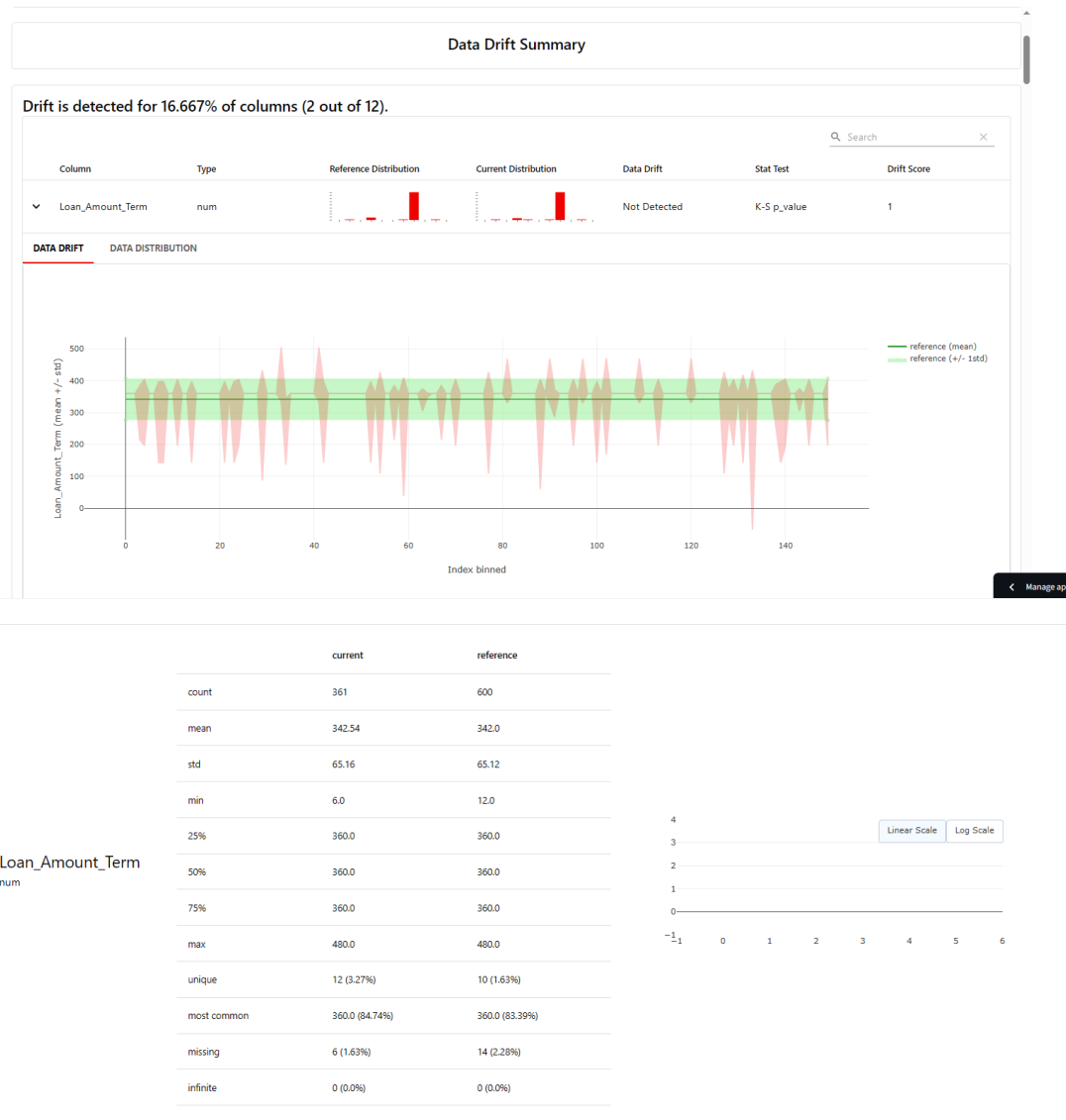


- **Figure 35.** Distribution comparison for CoapplicantIncome.



- **Figure 36.** Distribution comparison for LoanAmount.

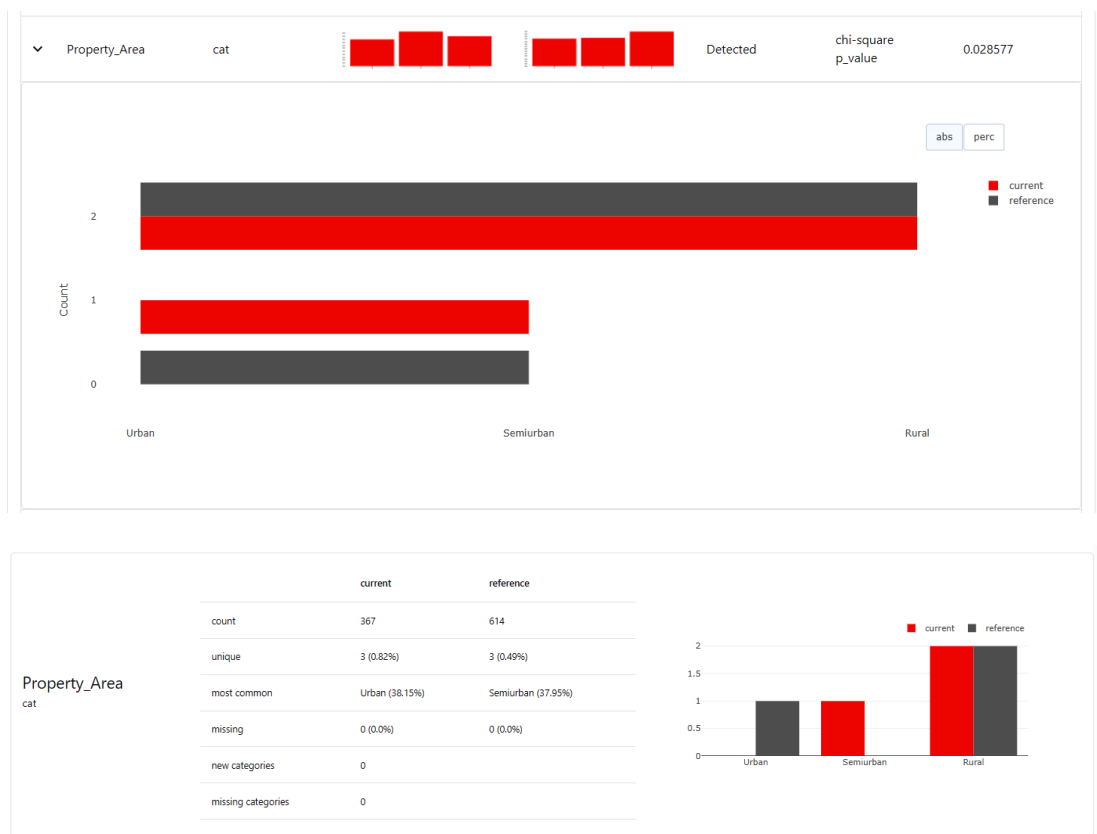
Data Drift Report



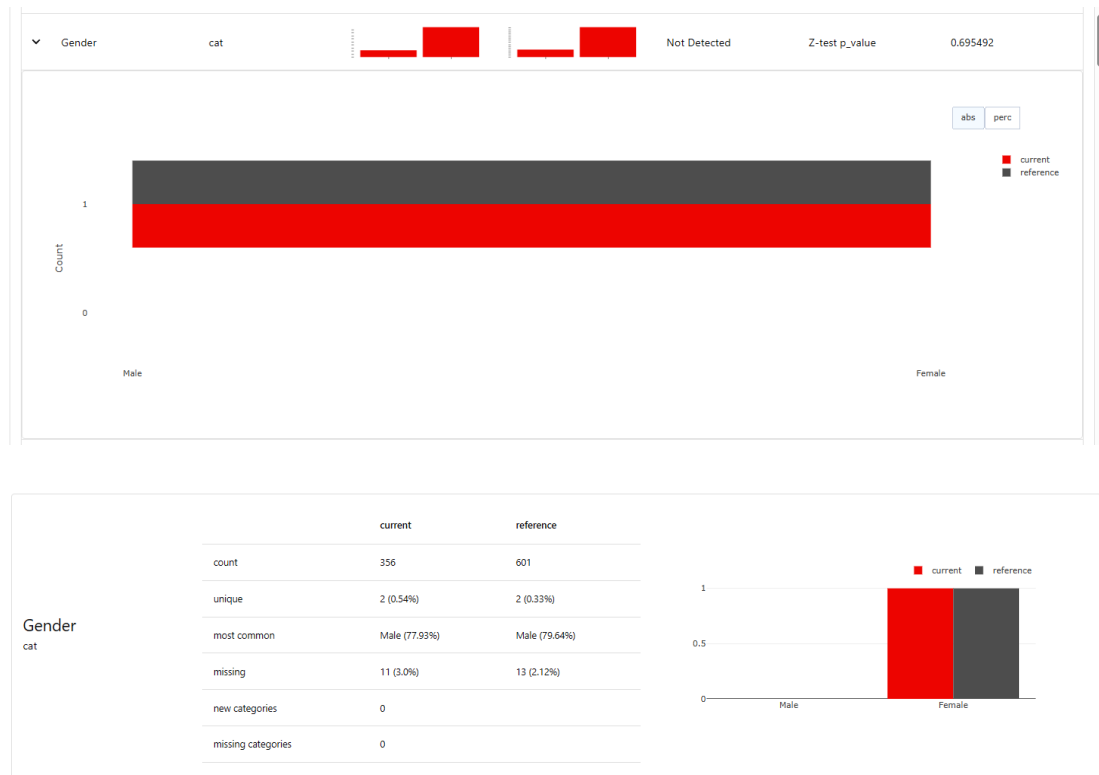
- **Figure 37.** Distribution comparison for Loan_Amount_Term.



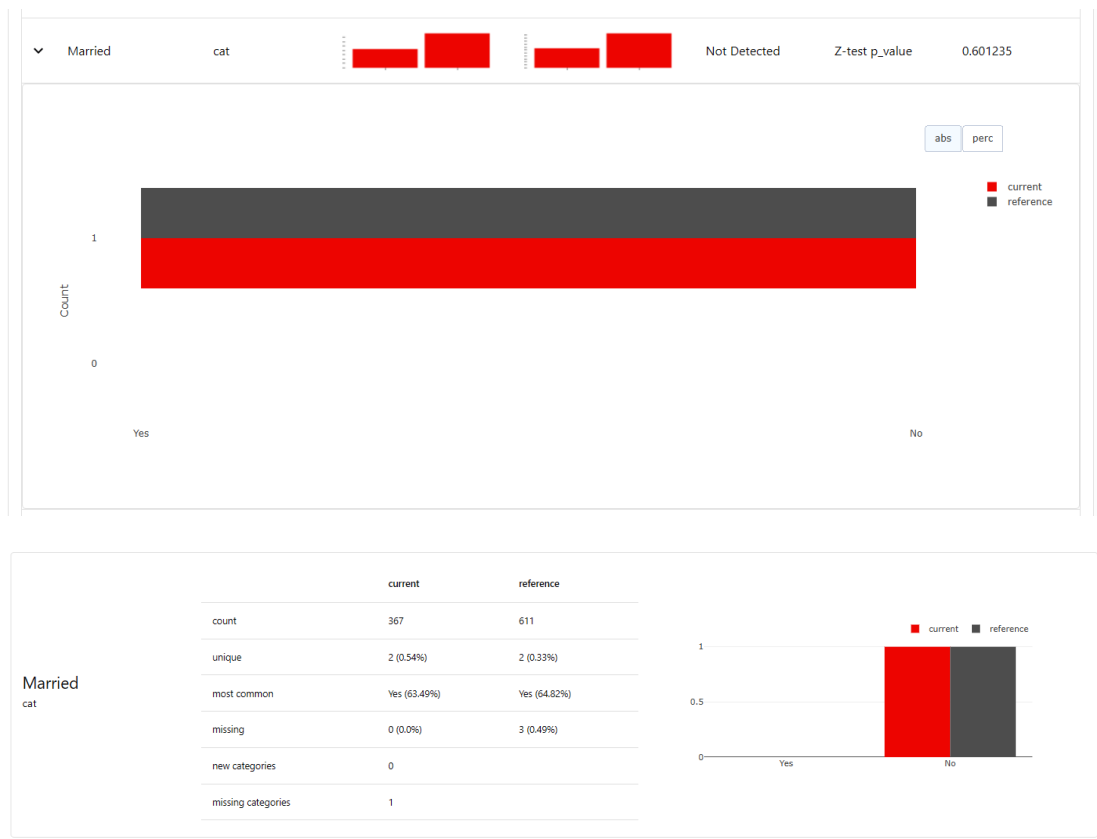
- **Figure 38.** Distribution comparison for Credit_History.



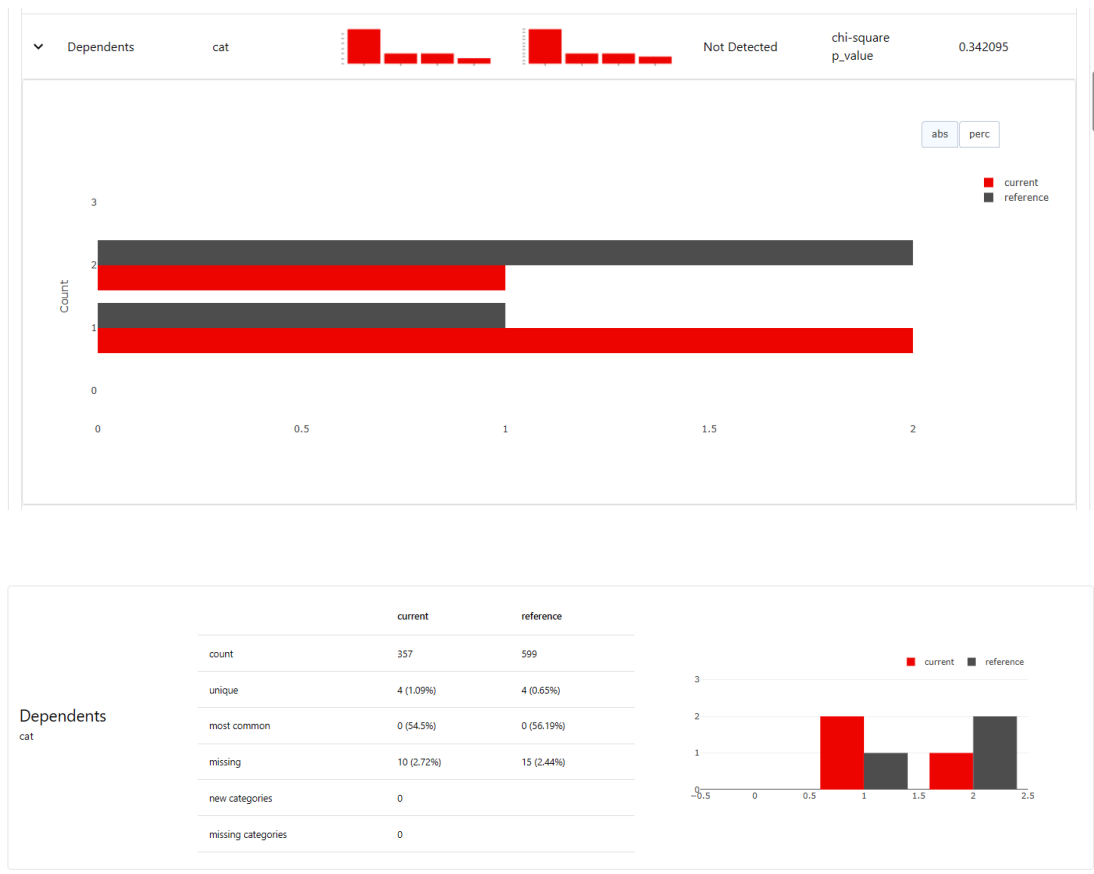
- **Figure 39.** Distribution comparison for Property_Area.



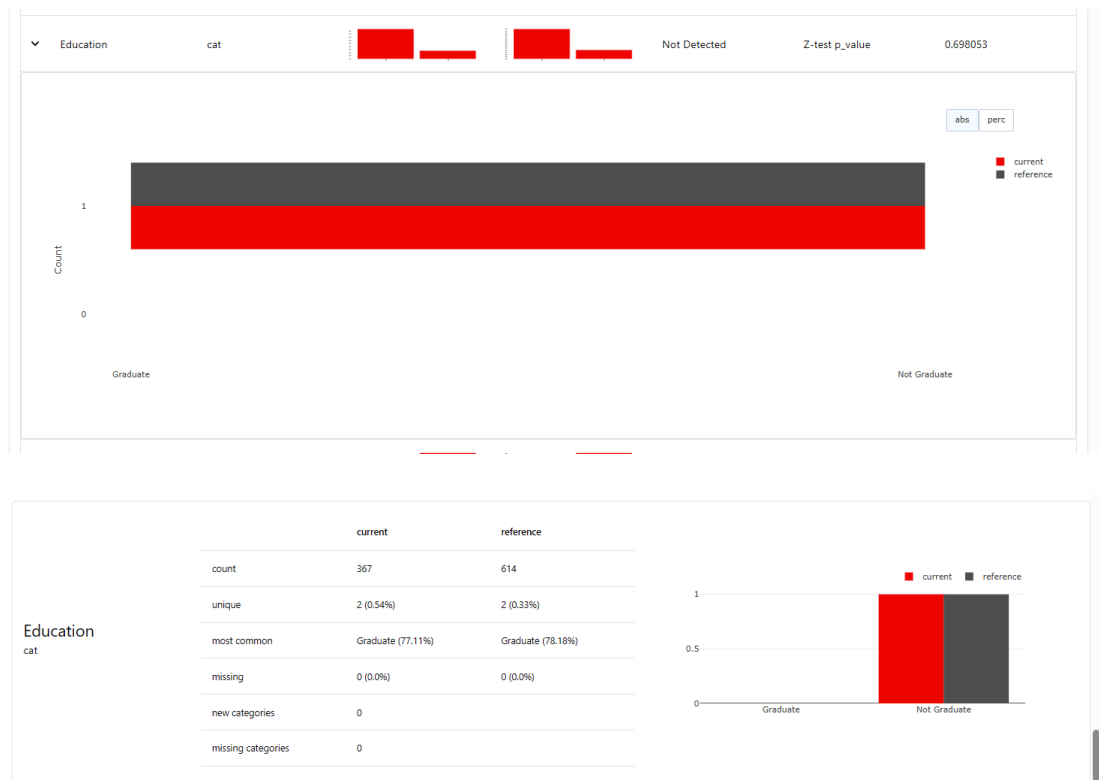
- **Figure 40.** Distribution comparison for Gender.



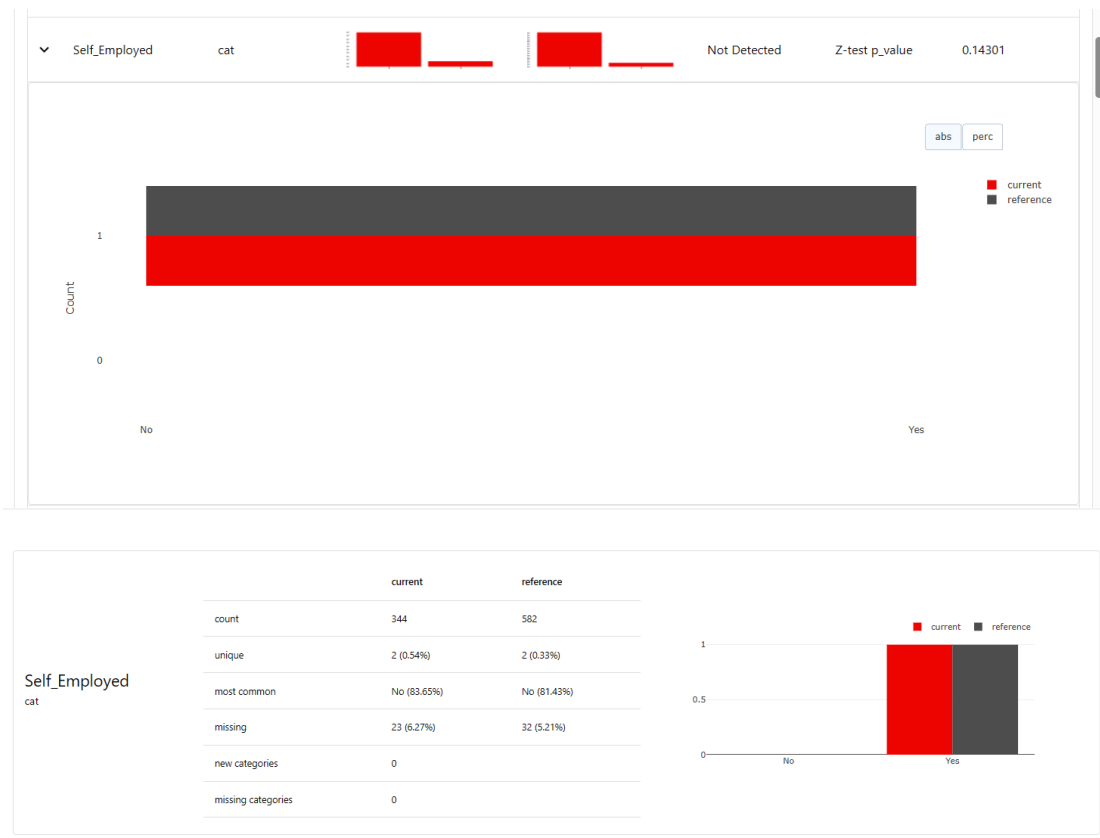
- **Figure 41.** Distribution comparison for Married.



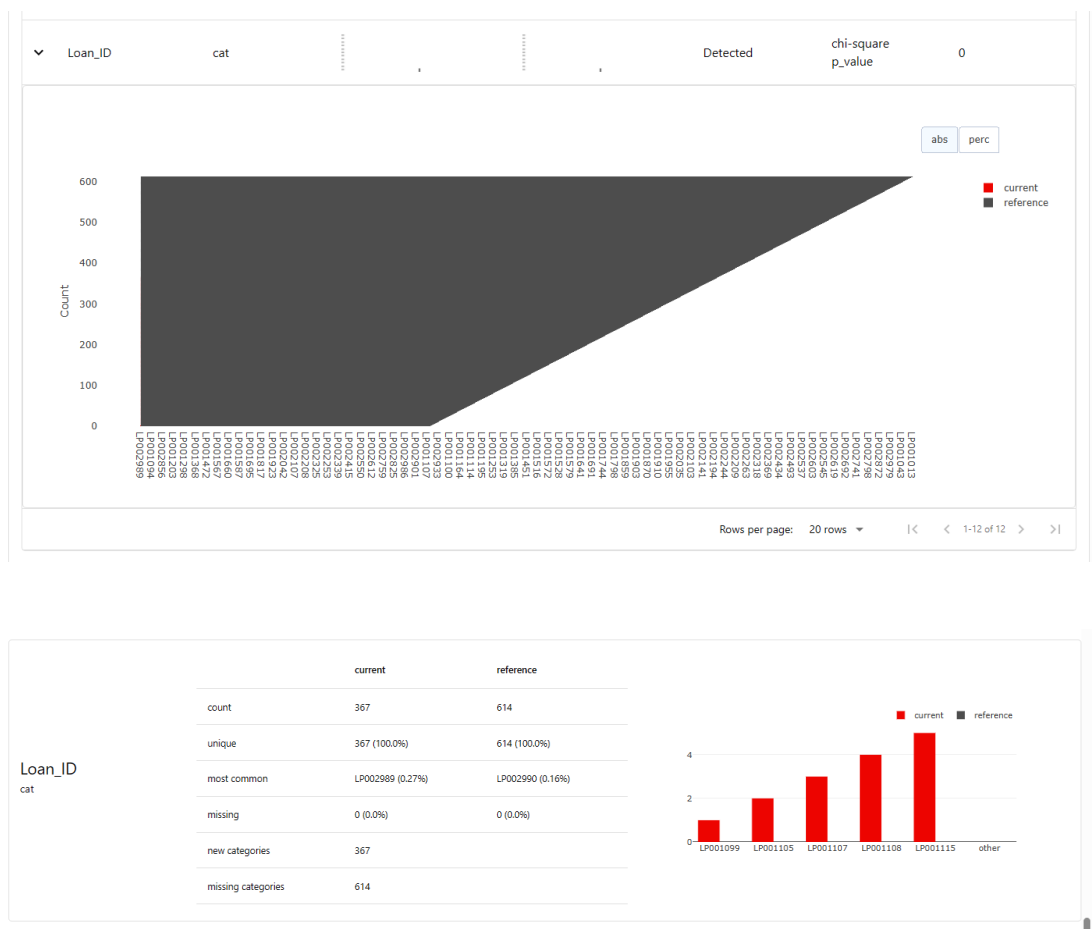
- **Figure 42.** Distribution comparison for Dependents.



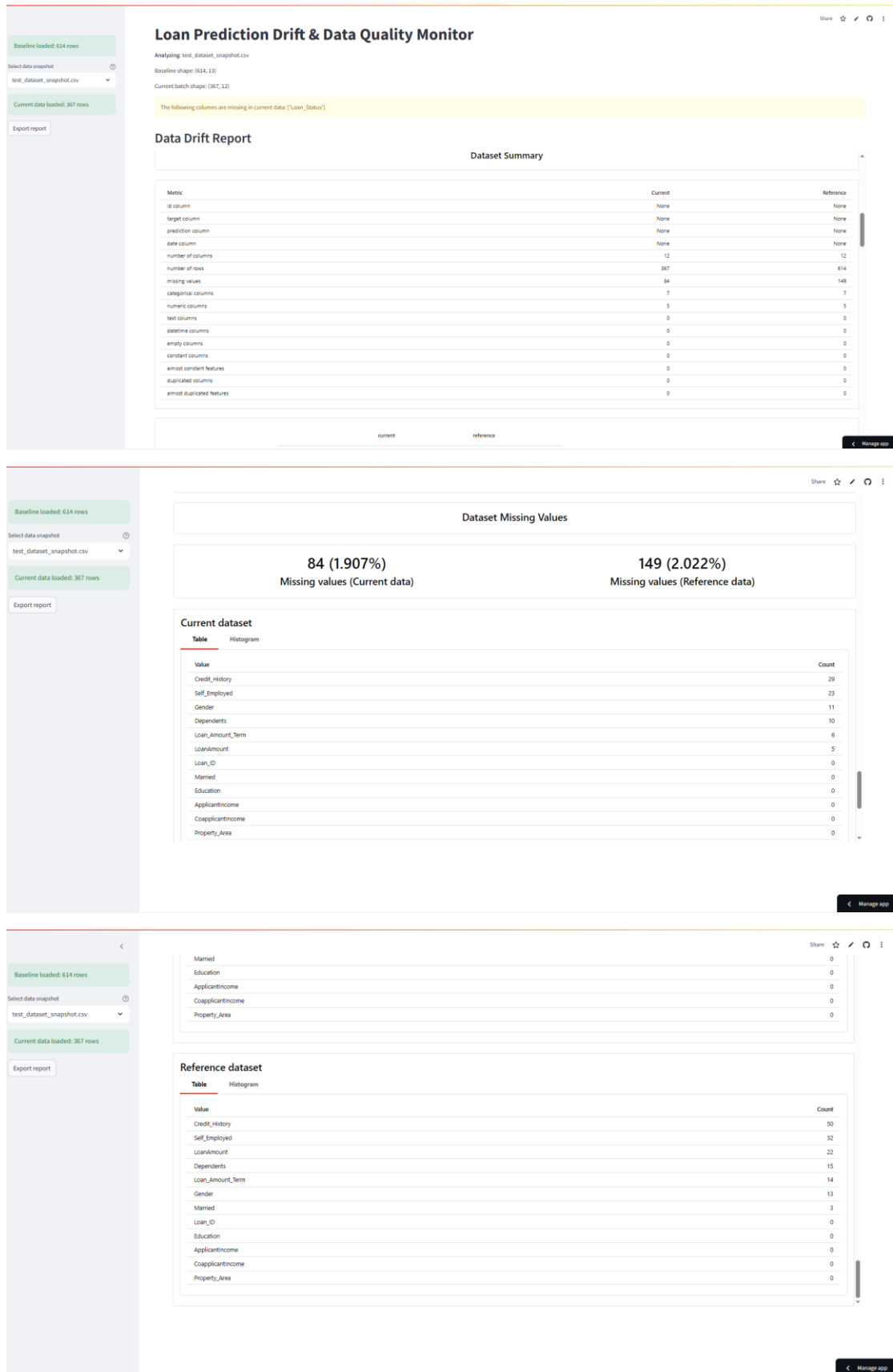
- **Figure 43.** Distribution comparison for Education.



- **Figure 44.** Distribution comparison for Self_Employed.



- **Figure 45.** Additional feature-level panel (e.g. combined view).



- **Figures 46.** Further Evidently panels showing additional summary tables, confidence scores, and diagnostic views (for example, feature importance in

drift, sample-level contributions, or alternative visual layouts of the same drift information).

Together, Figures 31–46 provide a detailed visual record of how the reference training dataset and the current production batch compare for each feature.

10. End-to-end Evidence Mapping

This section organises the figures into evidence groups that connect the offline training environment, model registry, online service, and drift monitoring.

10.1 Evidence Group 1: MLflow and model provenance (Figures 2–20)

Figures 2–8 show the experiments and individual training runs recorded in MLflow. They demonstrate that:

- each run has logged hyperparameters, metrics, and artefacts, and
- experiments are structured under a consistent naming scheme (e.g. `loan_prediction_model`).

Figures 9–12 show the Model Registry, where the best runs are promoted to named versions and stages. Figures 13–20 then illustrate why a particular model (e.g. LoanPrediction version 11, XGBoost) is considered the best candidate for production based on its metrics and diagnostic plots.

10.2 Evidence Group 2: Online inference on EKS (Figures 21–29)

The Prometheus Figures 21–26 show that the `/metrics` endpoint is being scraped and that key metrics such as `model_predictions_total`, `http_requests_total{path="/prediction_api"}`, and pod health status are available.

The Grafana dashboards (Figures 27–28) provide a higher-level overview of system behaviour, including request rate, latency, error rate and resource usage.

Finally, Figure 29 shows Kubernetes pod logs where prediction requests are processed by **model version 11** in the Production stage. These logs connect the registry information with live traffic.

10.3 Evidence Group 3: K8s batch → dataset files (Figures 1 and 30)

Figure 30 shows the script `collect_k8s_predictions.py` sending five requests to the FastAPI endpoint and saving the results to `k8s_latest_batch.csv`. Figure 1 shows the resulting CSV-style data, which includes IDs K8S_001–K8S_005 and the associated feature values.

This proves that the current batch used for drift analysis is derived directly from the online service running on EKS.

10.4 Evidence Group 4: Drift dashboard on Evidently (Figures 31–46)

Figures 31–32 confirm that the Evidently app successfully loads a baseline dataset of 614 rows and a current batch of 5 rows with 13 columns, corresponding to the training data and K8s batch respectively.

Loan-Prediction_MLOps

Figures 33–46 show feature-level and overall drift analysis. They demonstrate that the system truly compares the training distribution against the live data, using the small batch collected from the Kubernetes-hosted service. Thus, these figures close the loop between training, deployment, monitoring and drift analysis.

11. Discussion and Future Work

11.1 Strengths of the current system

The system demonstrates several strong points:

- **Reproducible data and experiments:** DVC and MLflow ensure that datasets and models can be reproduced for any Git commit and run, which is important for debugging and audit.
- **Separation of concerns:** Data processing, training, inference, and monitoring are clearly separated into different modules and services.
- **End-to-end observability:** Prometheus and Grafana allow real-time monitoring of the API service, while Evidently adds visibility into data drift.

- **Evidence-based implementation:** The 46 figures create a transparent evidence trail from experiments and registry entries through to Kubernetes logs and drift dashboards.

11.2 Limitations

At the same time, several limitations remain:

- The size of the current batch for drift detection (five records) is very small, so drift conclusions must be interpreted with caution.
- The drift pipeline requires manual execution of `collect_k8s_predictions.py` and manual triggering of the Streamlit app. It is not yet fully automated.
- While the system supports rolling updates, advanced deployment strategies such as canary or blue–green deployments are not fully described or automated.
- The project focuses mainly on a single model; it does not yet include automated retraining or champion–challenger comparisons in production.

11.3 Future work

Future improvements could include:

- **Automated drift monitoring and alerts:** scheduling periodic batch collection and drift checks, and sending alerts via email or chat when drift exceeds a threshold.
 - **Canary or blue–green deployments:** integrating tools like Argo Rollouts or custom scripts to split traffic safely between model versions.
 - **Automated retraining:** triggering new training runs when significant drift is detected, and promoting new models through an automated approval process.
 - **Extended explainability and fairness dashboards:** leveraging SHAP and Fairlearn artefacts logged to MLflow and displaying them in Grafana or a dedicated UI.
-

12. Conclusion

This report has described a full MLOps system for a loan prediction task on the AWS platform. Starting from a supervised classification problem, the project builds a

pipeline that covers data versioning, training, experiment tracking, model registry, containerised deployment, monitoring, and drift detection.

The system shows how classical machine learning components—such as scikit-learn pipelines, XGBoost models and JSON-based FastAPI endpoints—can be integrated with modern MLOps tools like DVC, MLflow, Kubernetes, Prometheus, Grafana and Evidently. Through 46 figures, the documentation provides concrete evidence that the deployed service runs the registered production model, that its metrics are monitored in real time, and that its input data can be compared with the training distribution for drift.

Overall, the project demonstrates a practical blueprint for transforming a typical machine learning prototype into a production-ready, observable and maintainable MLOps solution.

Appendices

Appendix A. Key CLI Commands (Selection)

This appendix summarises some of the most important commands used in the project.

A.1 DVC and datasets

```
# Configure AWS credentials (once)
aws configure
```

```
# Pull datasets from S3 via DVC
dvc pull
```

```
# Track new or updated datasets
dvc add datasets/
dvc push
```

A.2 MLflow and experiments

```
# Sync mlruns from S3
aws s3 sync s3://loanpred-mlops-20251118-120330/mlruns/ ./mlruns/
```

```
# Start MLflow UI
mlflow ui --backend-store-uri file:./mlruns --port 5000
```

```
# Train and log models
python -m prediction_model.training_pipeline
```

A.3 Docker and local API

```
# Build image
docker build -t loan-prediction:local .
```

```
# Run image locally
docker run -p 8005:8005 \
  -e MLFLOW_TRACKING_URI="http://host.docker.internal:5000" \
  loan-prediction:local
```

A.4 Kubernetes and EKS

```
# Update kubeconfig for EKS
aws eks update-kubeconfig --name loan-prediction-eks --region eu-west-2

# Apply Kubernetes manifests
kubectl apply -f k8s/namespace.yml
kubectl apply -f k8s/deployment.yml
kubectl apply -f k8s/services.yml

# Port-forward FastAPI service
kubectl port-forward -n loan-prediction-mlops svc/loan-prediction-service 8005:8005
```

A.5 Monitoring (Prometheus and Grafana)

```
# Local monitoring with docker-compose
cd monitoring
docker-compose up -d

# Port-forward Prometheus on EKS
kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus
9090:9090

# Port-forward Grafana on EKS
```

```
kubectrl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3001:80
```

A.6 Drift monitoring and batch collection

```
# Collect predictions from K8s endpoint
```

```
python collect_k8s_predictions.py
```

```
# Run Evidently Streamlit app
```

```
cd drift_monitoring
```

```
streamlit run app_v1.py
```

Appendix B. Selected Configuration Snippets

This appendix can include, for example, the `.dvc/config`, Prometheus `scrape_configs`, and shortened Kubernetes manifests, to support the architectural descriptions in the main text.